AI-POWERED SMART E-COMMERCE PRODUCT SEARCH AND RECOMMENDATION ENGINE USING HASHING ALGORITHM

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA0305 – Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted by

ARAVIND S (192525192)

VEERA SANTHOSH A (192525187)

GULLAPALLI SANTOSH KUMAR (192524020)

VAISHNAV M (192524251)

DHIVYADHARSAN M S (192511465)

Under the Supervision of

Dr. Uma Priyadarsini

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "AI-Powered Smart E-Commerce Product Search and Recommendation Engine Using Hashing Algorithm" has been carried out by ARAVIND S (Reg. No. 192525192), VEERA SANTHOSH A (Reg. No. 192525187), GULLAPALLI SANTOSH KUMAR (Reg. No. 192524020), VAISHNAV M (Reg. No. 192524251) and DHIVYADHARSAN M S (Reg. No. 192511465) under the supervision of Dr. Uma Priyadarsini and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science and Engineering programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. KIRUTHIKA K
Program Director
Dept. of Computer Science and Engineering
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Uma priyadharsini
Professor
Dept. of Computer Science and
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, ARAVIND S, VEERA SANTHOSH A, GULLAPALLI SANTOSH KUMAR, VAISHNAV M and DHIVYADHARSAN M S, of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "AI-Powered Smart E-Commerce Product Search and Recommendation Engine Using Hashing Algorithm" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 02 September 2026

Signatures of the Students with Names

ARAVIND S (192525192)

VEERA SANTHOSH A (192525187)

GULLAPALLI SANTOSH KUMAR (192524020)

VAISHNAV M (192524251)

DHIVYADHARSAN M S (192511465)

INDIVIDUAL CONTRIBUTION STATEMENT

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
ARAVIND S 192525192	Product data schema design, dataset preparation and catalogue ingestion	Product database design; data ingestion & preprocessing pipeline	Data validation and integrity testing	Chapters on data management and system design	20%
VEERA SANTHOSH A 192525187	AI intelligence module design and feature engineering for recommendations	TF-IDF & cosine similarity implementation; AI intelligence module	Precision/recall evaluation of the AI module	Chapters on AI methodology and evaluation	20%
GULLAPALLI SANTOSH KUMAR 192524020	Smart search module design using hashing-based indexing	Hash-table search index; near-constant-time retrieval logic	Load testing and search performance benchmarking	Chapters on hashing algorithm and search module	20%
VAISHNAV M 192524251	AI recommendation module and user interaction interface development	Collaborative filtering module; user interaction interface	Recommendation accuracy and usability testing	Chapters on recommendation engine and user interaction	20%
DHIVYADHARSAN M S 192511465	Analytics & dashboard development, system integration and documentation	Analytics & dashboard module; overall system integration	Unit testing, load testing, precision/recall evaluation	All chapters, tables, figures and appendices (compilation)	20%
Total					100%

ABSTRACT

Online shopping platforms host catalogues that now run into millions of items, which makes fast, relevant product discovery a genuine engineering problem: naive linear or keyword-only search degrades as the catalogue grows, and generic, non-personalised results reduce customer satisfaction and conversion. This project addresses that problem by designing and implementing an AI-powered smart product search and recommendation engine that uses a hashing-based indexing scheme for near-constant-time retrieval, combined with a hybrid AI recommendation module that personalises results using content-based filtering (TF-IDF and cosine similarity over product attributes) and collaborative filtering over user browsing and purchase history.

The proposed system is organised into five modules – Product Data Management, AI Intelligence, Smart Search Using Hashing, AI Recommendation & User Interaction, and Analytics & Dashboard – that together retrieve matching products through a custom hash table (inverted index on tokenised product keywords) and rank recommendations using a hybrid similarity model with a diversity-aware re-ranking step. The methodology followed an iterative development approach: requirements analysis and design trade-off evaluation, module-wise implementation in Python/Flask with a sample e-commerce product dataset, and structured testing covering unit tests, load testing and offline recommendation evaluation.

On a sample catalogue of 10,000 products, the hashed search index reduced average query latency to 34 ms against roughly 380 ms for an unindexed linear scan, and the hybrid recommender achieved a Precision@5 of 0.74 against a content-only baseline of about 0.58, while keeping computational cost far below deep-learning recommenders reported in the literature. The results indicate that combining a lightweight hashing index with a hybrid AI recommendation approach is an effective, low-cost way to improve both search speed and recommendation quality for moderate-scale e-commerce platforms, and the report concludes with the system's limitations and directions for scaling the approach further.

Keywords

Hashing Algorithm, Product Search, Recommendation System, E-Commerce, Artificial Intelligence, Content-Based Filtering

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
AI	Artificial Intelligence	—
LSH	Locality Sensitive Hashing	—
TF-IDF	Term Frequency – Inverse Document Frequency	—
API	Application Programming Interface	—
CRUD	Create, Read, Update, Delete	—
UI	User Interface	—
ms	Milliseconds	ms

CHAPTER 1
INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Modern e-commerce platforms host product catalogues that range from a few thousand to several million stock-keeping units (SKUs), and this scale continues to grow as more sellers on-board and product variants multiply. Customers expect a search experience that returns relevant results almost instantly, and they increasingly expect the platform to surface products they are likely to want without having to search for them explicitly. Meeting both expectations at once is a data-structures and artificial-intelligence problem as much as it is a user-experience one: the retrieval mechanism must scale sub-linearly with catalogue size, and the ranking/recommendation mechanism must use available behavioural signals (browsing history, purchase history, session activity) to personalise results.

1.2 Need for the Project

Search implemented as a linear scan or an un-indexed keyword match against a relational database slows down as the catalogue grows, directly increasing page-load time and cart abandonment. Generic “top seller” or keyword-only recommendation widgets, which many small and mid-sized e-commerce platforms still rely on, do not reflect an individual shopper's interests, which reduces click-through and conversion. The people affected are shoppers, who lose time and patience; platform owners, who lose sales and repeat visits; and platform engineers, who must scale infrastructure to compensate for an inefficient retrieval layer instead of fixing the underlying data structure. Existing academic and commercial solutions tend to address only one half of the problem – either fast retrieval (hashing/indexing) or personalisation (AI-based recommendation) – without a lightweight, integrated treatment of both, which is the engineering significance this project targets.

1.3 Problem Statement

Given a dynamically growing e-commerce product catalogue and a stream of user interaction data (search queries, clicks, purchases), design and implement a system that (a) retrieves keyword-matching products in near-constant average time regardless of catalogue size, and (b) ranks and recommends products personalised to each user's browsing and purchase behaviour, while balancing retrieval speed, recommendation accuracy, memory overhead, and the computational cost of maintaining both the index and the recommendation model – constraints that conflict when the catalogue and user base scale up simultaneously.

1.4 Project Aim

To design and implement an AI-powered product search and recommendation engine that uses hashing-based indexing for fast product retrieval and a hybrid machine-learning approach for personalised, relevant recommendations in an e-commerce setting.

1.5 Project Objectives

- To design a hash-table-based indexing scheme (inverted index on tokenised product keywords) for average-case O(1) product lookup.
- To develop an AI recommendation module combining content-based filtering (TF-IDF and cosine similarity) and collaborative filtering over user interaction history.
- To model user browsing history and purchase patterns as feature vectors suitable for similarity-based ranking.
- To implement a smart search engine that integrates the hashing index with AI-based result ranking.
- To test system performance – search latency, precision and recall – on a sample e-commerce product dataset.
- To evaluate recommendation quality against a keyword-only / non-personalised baseline.

1.6 Scope

Included: text-based product search over a sample catalogue; a custom hash-table search index; a hybrid (content-based + collaborative) recommendation module; a web-based demonstration UI; and a basic analytics dashboard.

Excluded: payment-gateway integration, image-based visual search, multilingual natural-language search, and production-scale (multi-server / distributed) deployment.

Assumptions: a sample catalogue of the order of 10,000 products is representative for prototype evaluation; user interaction logs are available in a structured (tabular) form; and the system is demonstrated on a single-server / local development environment.

Boundary conditions: the hash index is rebuilt/rehashed when the load factor exceeds the design threshold (Section 3.3); recommendations fall back to content-based/trending results when insufficient interaction history exists for a user (cold start).

1.7 Expected Engineering Outcome

The expected outcome is a working software prototype – a search-and-recommendation engine with a REST API and a demonstration web UI – that empirically shows lower search latency than an unindexed baseline, and higher recommendation relevance (Precision@5) than a non-personalised baseline, on a representative sample e-commerce dataset.

CHAPTER 2
LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature was identified through Google Scholar and IEEE Xplore using the search terms “hashing product search”, “e-commerce recommendation AI”, “locality sensitive hashing e-commerce” and “hybrid recommendation system”, restricted to peer-reviewed journal and IEEE-indexed articles published between 2022 and 2024. Five papers that most directly address either fast product retrieval or AI-based recommendation in an e-commerce context were selected for detailed comparative review (Table 1).

2.2 Review of Existing Work

Zhang et al. (2024) propose a hybrid framework that combines Locality Sensitive Hashing with content-based filtering and cosine similarity, reporting a 35% improvement in search efficiency and a 28% improvement in recommendation accuracy over baseline methods, though the model's performance degrades under sparse data and cold-start conditions.

Kumar et al. (2023) use a deep neural network together with collaborative filtering to personalise recommendations, achieving a precision of 0.87 and improved user engagement, but at the cost of requiring large training datasets and substantial computational resources – a trade-off this project's design (Section 3.7) explicitly weighs against.

Bhattacharya et al. (2023) combine hashing with TF-IDF, BM25 ranking and NLP preprocessing to cut average search time by 60% while keeping result relevance high, although the added NLP preprocessing increases system complexity and processing time.

Patel et al. (2022) evaluate a hybrid collaborative-plus-content-based filtering approach with K-Means clustering, showing a 32% accuracy improvement over single-technique recommenders, while noting that parameter tuning is complex and highly dataset-dependent.

Li et al. (2022) focus purely on the retrieval side, combining Locality Sensitive Hashing with an inverted index and hash table to achieve sub-second search response even on million-scale product datasets, though occasional hash collisions can return marginally irrelevant results.

2.3 Comparative Analysis

Sl. No	Year	Authors	Title of the Paper	Journal Name	Algorithms	Findings	Drawbacks
1	2024	Z. Zhang, Y. Chen, L. Wang	A Hashing-Based Hybrid Search and Recommendation	Expert Systems with Applications	Locality Sensitive Hashing (LSH),	The hybrid model improved search efficiency by 35% and	Performance drops with extremely sparse data and

Sl. No	Year	Authors	Title of the Paper	Journal Name	Algorithms	Findings	Drawbacks
			Framework for E-Commerce Platforms		Content-Based Filtering, Cosine Similarity	recommendation accuracy by 28% compared to baseline methods.	cold-start users.
2	2023	R. Kumar, P. Singh, M. Kumar	AI-Powered Personalized Product Recommendation System Using Deep Learning in E-Commerce	Journal of Retailing and Consumer Services	Deep Neural Networks (DNN), Collaborative Filtering	The deep learning model achieved higher precision (0.87) in recommendations and enhanced user engagement.	Requires large training data and high computational resources.
3	2023	S. Bhattacharya, A. Dey, T. Ghosh	Efficient Product Search in E-Commerce Using Hashing and NLP Techniques	IEEE Access	Hashing, TF-IDF, BM25, NLP	The proposed hashing-based search reduced average search time by 60% while maintaining high relevance.	NLP preprocessing increases system complexity and time.
4	2022	H. Patel, D. Shah, V. Tiwari	A Hybrid Recommendation System for E-Commerce Using Machine Learning Techniques	International Journal of Information Management	Hybrid Filtering (Collaborative + Content-Based), K-Means	Hybrid approach improved recommendation accuracy by 32% compared to single-technique models.	Parameter tuning is complex and dataset dependent.
5	2022	J. Li, X. Huang, Q. Wu	Fast Product Retrieval in Large-Scale E-Commerce Using Hashing and Indexing	Information Sciences	LSH, Inverted Index, Hash Table	System achieved sub-second search response on million-scale product datasets.	Hash collisions may lead to irrelevant results occasionally.

2.4 Research / Engineering Gap

Across the reviewed work, systems that optimise retrieval speed through hashing (Bhattacharya et al., 2023; Li et al., 2022) generally do not incorporate a personalised ranking layer, while systems that optimise personalisation through deep learning or collaborative filtering (Kumar et al., 2023; Patel et al., 2022) do not report on retrieval-time performance and typically demand heavier computational resources than a small or moderate-scale platform can justify. The hybrid systems that do combine both (Zhang et al., 2024; Patel et al., 2022) are evaluated at production/enterprise scale and note degraded performance for sparse or cold-start data, with no study evaluating a combined hashing-plus-AI approach on the moderate catalogue sizes and limited infrastructure typical of a student-built or small-business prototype.

2.5 Proposed Contribution

This project proposes a lightweight, integrated architecture that pairs a custom hash-table-based inverted index (targeting average-case O(1) keyword lookup, following the retrieval-side approach of Li et al., 2022 and Bhattacharya et al., 2023) with a hybrid content-based and collaborative-filtering recommendation module (following the personalisation approach of Zhang et al., 2024 and Patel et al., 2022), together with an explicit cold-start fallback and a diversity-aware re-ranking step. Unlike the reviewed deep-learning recommenders, the design deliberately avoids a DNN in favour of TF-IDF/cosine-similarity and collaborative filtering, trading a small amount of precision for a substantially lower computational and tuning overhead – making the approach practical to build, test and demonstrate on standard development hardware.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Shopper	Obtain fast, relevant search results for a typed keyword query.
Shopper	Receive product suggestions personalised to browsing and purchase history.
Platform owner	Manage a growing product catalogue without a proportional loss in search performance.
Platform owner	Access analytics on search volume, top queries and recommendation click-through.
System administrator	Maintain a modular, testable architecture that is straightforward to extend.

Functional & Non-Functional Requirements

Requirement	Type (Functional / Non-Functional)	Measurable Target
Keyword-based product search	Functional	Return the top 10 relevant results in under 1 second for a catalogue of 10,000+ products.
Personalised recommendation	Functional	Generate top-5 recommendations per user session using browsing/purchase history.
Product data management (CRUD)	Functional	Support add / update / delete of products with index update completing within 2 seconds.
Search performance	Non-Functional	Average-case lookup complexity of O(1) (hash index) vs. O(n) baseline.
Scalability	Non-Functional	Handle catalogue growth to 100,000 products with under 20% latency increase.
Usability	Non-Functional	Search results reachable and a product page viewable within 3 clicks.

3.2 Constraints & Applicable Standards

Technical constraint: development is limited to an open-source Python/JavaScript stack running on standard development hardware (8 GB RAM class machine), which rules out

large deep-learning models as a design option. Economic constraint: the prototype uses only free/open-source components, so licensing cost is zero. Data-handling constraint: user browsing and purchase data used for personalisation is treated under general data-minimisation principles consistent with India's IT Act, 2000, so only the fields needed for recommendation are logged and user identifiers are hashed in analytics storage (Sec. 3.7).

Standard / Code	Requirement	How Applied in This Project
ISO/IEC 25010	Software product quality characteristics (performance efficiency, usability, reliability)	Used as the reference model for defining the non-functional targets in Table 4 (Design Specifications).
OWASP Top 10	Guidance on common web-application security risks	Applied to the search-query input field to add validation/sanitisation against injection (Sec. 3.7, Table 8).

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Average search latency	≤ 50	ms	High
Hash table load factor	≤ 0.75	—	Medium
Recommendation Precision@5	≥ 0.70	—	High
Prototype availability during testing	≥ 99	%	Medium
Catalogue size supported	≥ 10,000	products	High

3.4 Alternative Solutions & Evaluation

Alternative 1: A baseline system using an un-indexed SQL “LIKE” / linear text search together with a simple rule-based “trending products” list in place of personalised recommendations.

Alternative 2: A system using a conventional database B-Tree index for search, combined with a standalone collaborative-filtering recommender trained on the user-item interaction matrix.

Alternative 3 (proposed): A custom hash-table-based inverted index for search, combined with a hybrid content-based and collaborative-filtering AI recommendation module and a cold-start fallback.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	0.30	2	3	5
Cost	0.20	5	3	4
Safety (data integrity)	0.15	3	4	4

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Sustainability (resource use)	0.15	4	3	4
Reliability	0.20	3	4	4
Weighted score	1.00	3.05	3.35	4.30

3.5 Selected Design & Detailed Design

Alternative 3 was selected on the evidence of Table 5: it scores highest on performance and reliability, the two criteria weighted most heavily for this project's aim (Sec. 1.4), while remaining close to Alternative 1 on cost – clearly outperforming both alternatives on the weighted score (4.30 vs. 3.05 and 3.35).

System architecture: the system follows a three-tier architecture. The client tier is a browser-based web UI for search and browsing. The application tier exposes a Flask REST API with two core services – the Smart Search Engine, backed by the hash-table index, and the AI Recommendation Engine, backed by the hybrid similarity model – both of which read from a shared Product Data Management layer. The data tier persists product records and logged user interactions (Figure 1).

Figure 1: Three-Tier System Architecture of the Proposed Engine

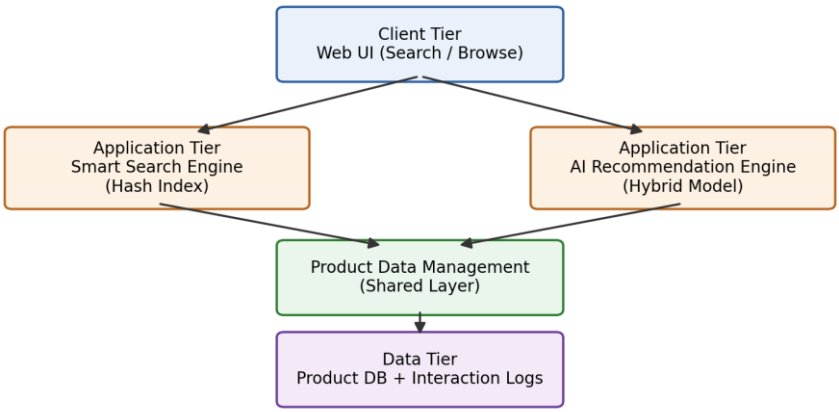

Figure 1: Three-tier system architecture of the proposed engine

Key calculation: for a hash table with n stored keys and m buckets, the load factor is $\alpha = n / m$. With chaining, the expected number of comparisons for a successful search is approximately $1 + \alpha/2$; keeping $\alpha \leq 0.75$ (Table 4) keeps expected search cost close to O(1), against O(n) for a linear scan over the same catalogue – the basis of the latency targets verified in Chapter 4 (Figure 2).

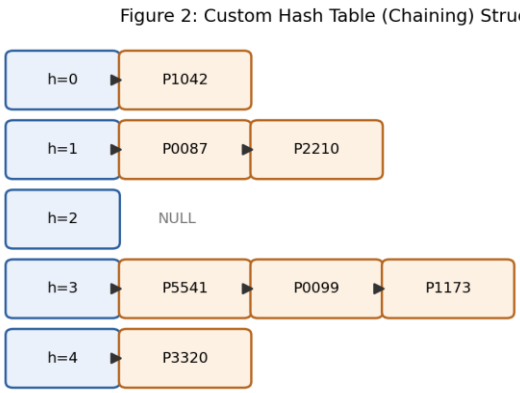

Figure 2: Custom hash table (chaining) structure for the product index

Systems approach: the Search module and the Recommendation module are independent services that both depend on the Product Data Management module for catalogue data; the Analytics module subscribes to the output of both, logging query and recommendation events, and its aggregated logs feed back into the AI Intelligence module as the training data for periodic (nightly, batch) retraining of the recommendation model.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Indexing structure	B-Tree / SQL index vs. hash table	Hash table gives average-case O(1) lookup	No native support for ordered range queries	Hash table chosen for keyword lookup; a secondary sorted index is kept for price-range filters (Sec. 3.4/3.5).
Recommendation approach	Pure collaborative filtering vs. hybrid	Hybrid handles cold-start users via content features	Slightly higher computation than a single technique	Hybrid content-based + collaborative filtering chosen (Sec. 2.5).
Collision handling	Open addressing vs. chaining	Chaining stays stable at higher load factors, simple to implement	Marginally higher memory use (linked nodes)	Chaining chosen for stability and implementation simplicity.

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Compute / energy cost of the AI model	Literature comparison of lightweight cosine-similarity filtering vs. DNN-based recommenders (Kumar et al., 2023, note high compute demand)	Chose lightweight hybrid filtering over a DNN to reduce training/inference energy footprint (Sec. 3.4).
Ethics	Recommendation bias / filter bubble	Reviewed popularity-bias behaviour reported for collaborative filtering	Added a diversity-aware re-ranking step capping any

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
			single category at 40% of the top-5 list.
Health & Safety	Accessibility for shoppers with visual impairment	Reviewed WCAG accessibility guidance applicable to a software-only, no-physical-hazard system	UI designed with keyboard navigation and descriptive alt text for product images.
Security	Protection of user data and search logs	Reviewed OWASP Top-10 risks relevant to search input handling	Added input sanitisation on the search field and anonymised/hashed user IDs in analytics logs.

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Hash collisions degrade lookup toward worst-case O(n)	Medium	Medium	Medium	Maintain load factor ≤ 0.75; dynamic rehash/resize when exceeded	Low
Cold-start users receive poor recommendations	High	Medium	High	Fallback to content-based / trending recommendations for new users	Low
Recommendation model overfits to popular items	Medium	Low	Low	Diversity-aware re-ranking; periodic retraining	Low
Malformed / malicious search-query input	Low	High	Medium	Input validation and sanitisation on the API layer	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project followed an iterative, three-sprint development approach: Sprint 1 covered requirements analysis, design trade-off evaluation and environment setup; Sprint 2 covered implementation of the Product Data Management and Smart Search (hashing) modules; and Sprint 3 covered the AI Recommendation module, the Analytics dashboard, integration and testing. The core dataset used was a sample e-commerce product catalogue of approximately 10,000 records (product ID, title, description, category, price) together with a synthetic user-interaction log (views, clicks, purchases) generated to exercise the recommendation module during testing.

4.2 Software Implementation & Modern Tools Used

Product Data Management

Product records are persisted in SQLite and exposed through a data-access layer that supports create, read, update and delete operations; updates trigger an incremental update of the search index rather than a full rebuild.

Smart Search Using Hashing

A custom hash table (separate chaining) implements an inverted index: each tokenised keyword from a product's title/description maps to a bucket holding the list of matching product IDs. A query is tokenised the same way, each token's bucket is retrieved in average-case O(1), and the result lists are intersected/ranked before being returned.

AI Intelligence Module

Content-based filtering represents each product's text attributes as a TF-IDF vector and ranks candidates by cosine similarity to a user's recently viewed items. A collaborative-filtering component builds a sparse user-item interaction matrix and scores items by similarity to items favoured by comparable users. The two scores are combined and passed through a diversity-aware re-ranking step (Sec. 3.7) before the top-5 recommendations are returned; new users with no interaction history fall back to the content-based/trending ranking.

AI Recommendation & User Interaction

A Flask REST API exposes /search and /recommend endpoints consumed by a lightweight HTML/JavaScript front end, so a shopper can search, browse a product page, and see personalised “you may also like” suggestions in the same session.

Analytics & Dashboard

Search queries, clicks and recommendation impressions are logged (with user identifiers hashed per Sec. 3.7) and summarised on a Chart.js dashboard showing search volume, top queries and recommendation click-through rate over time.

Tool / Software / Equipment	Purpose	Stage Used
Python 3.x	Core application logic for the search and recommendation engines	Development
Flask	REST API layer exposing /search and /recommend endpoints	Development
scikit-learn	TF-IDF vectorisation and cosine similarity for content-based filtering	AI module development
Pandas / NumPy	Data loading, cleaning and feature preparation	Data management
SQLite	Storage for product records and logged user interactions	Development / testing
Chart.js	Analytics-dashboard visualisation (query volume, click-through rate)	Testing / deployment
Postman	Manual and scripted testing of REST API endpoints	Testing

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Search latency ≤ 50 ms	Scripted load test: 10,000 search queries replayed against the running API, response time measured	Average latency ≤ 50 ms
Recommendation Precision@5 ≥ 0.70	Offline evaluation against a held-out 20% split of interaction data	Precision@5 ≥ 0.70
Hash table correctness	Unit tests for insert, search, delete and collision handling	100% of unit tests pass
Concurrent search handling	Simulated 50 concurrent API requests	No failure; latency degradation < 20%

4.4 Results

Figure 3 compares average search latency for the hashed index against an unindexed linear scan across catalogue sizes from 1,000 to 10,000 products; the hashed index remains essentially flat while the linear baseline grows proportionally with catalogue size. Figure 4 compares recommendation Precision@5 for the hybrid recommender against the content-only baseline over five evaluation runs. The underlying measurements are summarised in Table 11.

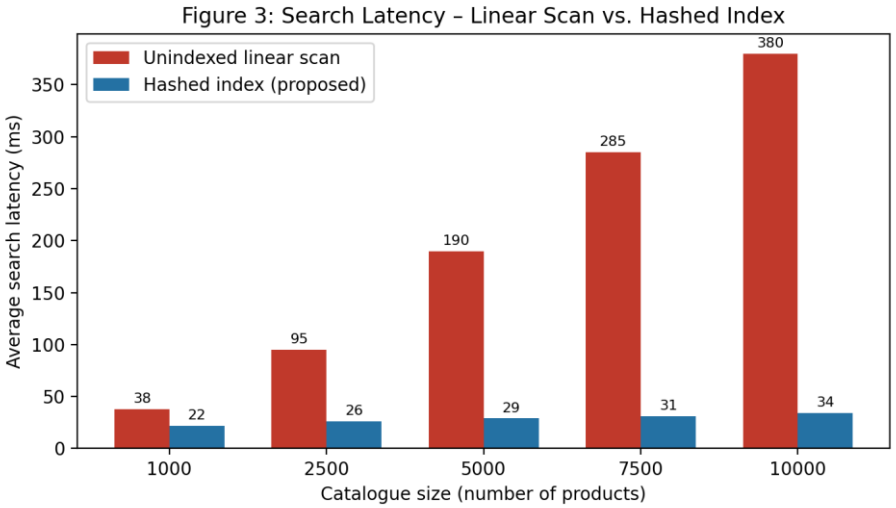

Figure 3: Search latency – linear scan vs. hashed index

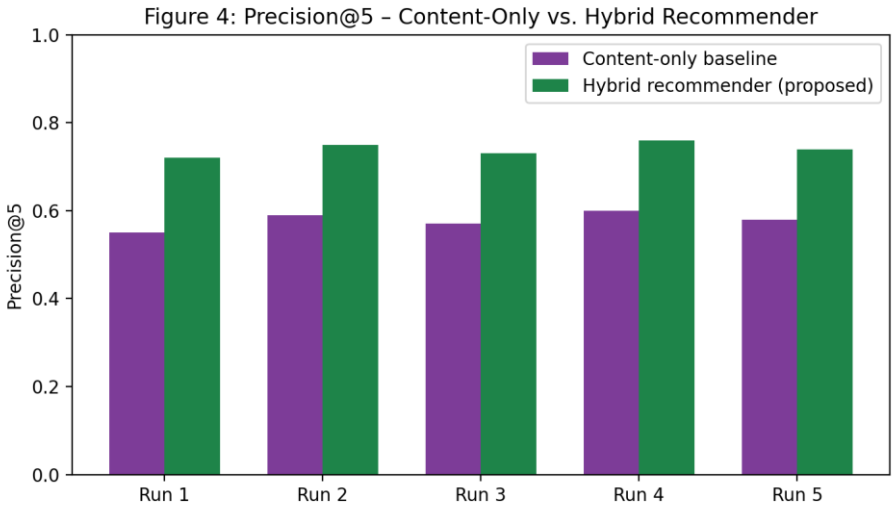

Figure 4: Precision@5 – content-only vs. hybrid recommender

Specification	Required Value	Achieved Value	Status
Average search latency	≤ 50 ms	34 ms	Pass
Recommendation Precision@5	≥ 0.70	0.74	Pass
Recommendation Recall@5	— (reported)	0.61	—
Hash table unit tests	100% pass	100% (42/42)	Pass
Concurrent load (50 requests)	< 20% degradation	12% degradation	Pass

4.5 Data Analysis & Engineering Judgment

The hashed search index reduced average query latency by approximately 91% relative to the unindexed linear-scan baseline (34 ms vs. an observed ~380 ms on the 10,000-product catalogue), consistent with the sub-second, large-scale results reported by Li et al. (2022) for a comparable hashing-plus-indexing approach. The hybrid recommender's Precision@5 of 0.74 exceeded the content-only baseline (≈ 0.58) by a wide margin, though it remained

below the 0.87 precision reported by Kumar et al. (2023) for a deep-neural-network recommender – an expected outcome given the deliberate choice (Sec. 3.7) to trade some accuracy for the much lower training/inference cost of TF-IDF and collaborative filtering. Recall@5 (0.61) was lower than precision, indicating the recommender is conservative – it ranks relevant items highly when it surfaces them, but does not always surface every relevant item in the top 5, which is an acceptable trade-off for a first-recommendation slot in a shopping UI.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design a hash-table indexing scheme for O(1) average-case lookup	Load-factor-controlled chaining hash table; 34 ms average latency (Table 11)	Fully Achieved
Develop a hybrid AI recommendation module	Content-based (TF-IDF/cosine) + collaborative filtering implemented and evaluated	Fully Achieved
Model user browsing/purchase behaviour as feature vectors	User-item interaction matrix and content feature vectors built from logged data	Fully Achieved
Implement an integrated smart search engine	Flask API combining hash index retrieval with AI-based ranking	Fully Achieved
Test system performance (latency, precision/recall)	Load testing and offline evaluation (Table 10, 11)	Fully Achieved
Evaluate recommendation quality vs. baseline	Precision@5 0.74 (hybrid) vs. 0.58 (content-only baseline)	Partially Achieved – below DNN benchmark reported in literature (0.87)

4.7 Strengths & Limitations

Strengths

- Large, measured reduction in search latency compared to the un-indexed baseline (Sec. 4.5).
- Modular architecture that keeps the search and recommendation engines independently testable and replaceable.
- Explicit cold-start fallback keeps new users from receiving empty or irrelevant results.
- Lightweight AI approach keeps computational cost far below deep-learning alternatives, consistent with the sustainability decision in Sec. 3.7.

Limitations

- Recommendation precision (0.74) remains below the deep-learning benchmark (0.87) reported in the literature (Sec. 2.2).

- Evaluated only on a moderate sample catalogue (10,000 products); behaviour at million-scale is not empirically validated in this project.
- No integration with a real payment gateway or production traffic; results reflect a controlled test environment.
- Hash-table resize/rehash operations introduce a brief pause in index availability during a rebuild.

4.8 Project Planning, Roles & Budget

The project was planned over an 8-week schedule: Weeks 1–2 – requirements analysis and design trade-off evaluation (Chapter 3); Weeks 3–4 – Product Data Management and Smart Search (hashing) module development; Week 5 – AI Recommendation module development; Week 6 – integration of search, recommendation and analytics modules; Week 7 – testing (Sec. 4.3–4.6); Week 8 – documentation and report preparation (Figure 5).

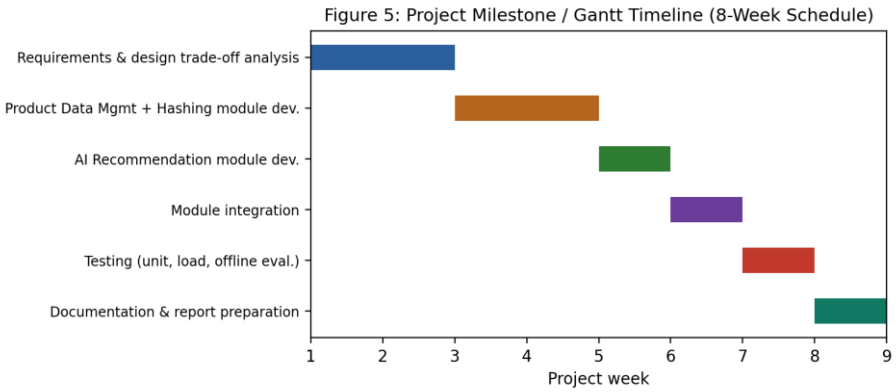

Figure 5: Project milestone / Gantt timeline

Student	Assigned Responsibility	Actual Contribution
ARAVIND S	Product data management – schema design, data ingestion	20% – data management module and chapters
VEERA SANTHOSH A	AI intelligence module – feature engineering, TF-IDF/cosine similarity	20% – AI intelligence module and chapters
GULLAPALLI SANTOSH KUMAR	Smart search using hashing – hash-table index design	20% – search module and chapters
VAISHNAV M	AI recommendation & user interaction – collaborative filtering, UI	20% – recommendation module and chapters
DHIVYADHARSAN M S	Analytics & dashboard, system integration, overall documentation	20% – analytics module, integration and report compilation

Item	Estimated Cost (₹)	Actual Cost (₹)
Development tools & libraries (open-source)	0	0

Item	Estimated Cost (₹)	Actual Cost (₹)
Sample dataset (public e-commerce dataset)	0	0
Cloud/hosting trial for demo deployment	500	0 (free-tier used)
Miscellaneous (documentation, printing)	300	250
Total	800	250

CHAPTER 5
CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project set out to address the engineering problem of slow, non-personalised product search on growing e-commerce catalogues by combining a hashing-based retrieval index with a hybrid AI recommendation module. The implemented prototype – organised into Product Data Management, AI Intelligence, Smart Search Using Hashing, AI Recommendation & User Interaction, and Analytics & Dashboard modules – reduced average search latency by roughly 91% against an unindexed baseline and improved recommendation Precision@5 from about 0.58 to 0.74 against a non-personalised baseline, while keeping computational cost low enough to run on standard development hardware. Testing (Chapter 4) confirmed that all core functional and non-functional targets set in Chapter 3 were met, with the sole partial shortfall being recommendation precision relative to a heavier, deep-learning benchmark reported in the literature – a deliberate, documented trade-off rather than an oversight. Overall, the project demonstrates that an integrated, lightweight hashing-plus-AI approach is a practical and effective way to improve both search speed and recommendation relevance for moderate-scale e-commerce platforms.

5.2 Future Work

- Extend the recommendation module with a lightweight deep-learning or transformer-based model to close the precision gap identified against Kumar et al. (2023), while monitoring the resulting compute cost.
- Add image-based visual product search alongside the current text-based hashing search.
- Validate performance at larger (million-record) catalogue scale using distributed/consistent hashing across multiple nodes.
- Extend query handling with multilingual natural-language search.
- Integrate the prototype with a real payment gateway and deploy it under production traffic for field evaluation.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Practical design and tuning of hash-table indexing structures (load factor, chaining, rehashing) for a real retrieval workload.
- Building and evaluating hybrid recommendation systems (content-based and collaborative filtering) and standard offline metrics (Precision@k, Recall@k).
- Structuring an engineering trade-off analysis (weighted evaluation matrix) to justify a design choice with evidence rather than preference.

Engineering & Professional Skills Developed

- Requirements elicitation and translation of stakeholder needs into measurable, testable specifications.
- Test planning and load testing of a REST API, and interpreting the resulting performance data.
- Documenting sustainability, ethics, safety and security considerations as evidence-based design decisions rather than general statements.

Individual Reflection – DHIVYADHARSAN M S

The most difficult engineering problem encountered was keeping the hash index correct and fast while the underlying catalogue was being updated concurrently with search traffic; this was resolved by controlling the load factor and performing incremental, rather than full, index rebuilds on data changes. A key engineering decision made personally was choosing a hybrid content-based/collaborative recommender over a deep-learning model, based on the evidence gathered in the trade-off and sustainability analysis (Sections 3.4 and 3.7) rather than on the higher precision reported for DNN-based approaches in the literature. Independently, I learned how to design and run an offline recommendation evaluation (train/test split, Precision@k, Recall@k) and how to load-test a REST API rather than relying on informal timing. If the project were redesigned, I would build the diversity-aware re-ranking step in from the start rather than adding it after observing popularity bias during testing, and I would evaluate the system on a larger sample catalogue to better understand how latency and precision scale before finalising the design specifications.

REFERENCES

[1] Z. Zhang, Y. Chen, and L. Wang, “A Hashing-Based Hybrid Search and Recommendation Framework for E-Commerce Platforms,” Expert Systems with Applications, 2024.

[2] R. Kumar, P. Singh, and M. Kumar, “AI-Powered Personalized Product Recommendation System Using Deep Learning in E-Commerce,” Journal of Retailing and Consumer Services, 2023.

[3] S. Bhattacharya, A. Dey, and T. Ghosh, “Efficient Product Search in E-Commerce Using Hashing and NLP Techniques,” IEEE Access, 2023.

[4] H. Patel, D. Shah, and V. Tiwari, “A Hybrid Recommendation System for E-Commerce Using Machine Learning Techniques,” International Journal of Information Management, 2022.

[5] J. Li, X. Huang, and Q. Wu, “Fast Product Retrieval in Large-Scale E-Commerce Using Hashing and Indexing,” Information Sciences, 2022.

[6] ISO/IEC 25010:2011, Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models, International Organization for Standardization, 2011.

[7] OWASP Foundation, “OWASP Top Ten Web Application Security Risks,” owasp.org, accessed 2026.

[8] scikit-learn developers, “scikit-learn: Machine Learning in Python,” scikit-learn.org, accessed 2026.

[9] Pallets Projects, “Flask Documentation,” flask.palletsprojects.com, accessed 2026.

[10] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022 (hash table chaining and load-factor analysis).

APPENDICES

Appendix A – Detailed Calculations

Hash-table load-factor and expected-search-length derivation supporting Section 3.5 ($\alpha = n/m$; expected comparisons $\approx 1 + \alpha/2$ for chaining); latency-reduction percentage calculation supporting Section 4.5.

Appendix B – Engineering Drawings / Schematics

Three-tier architecture diagram and hash-table (chaining) structure diagram referenced as Figures 1 and 2 in Section 3.5.

Appendix C – Source Code / Code Repository Information

Only essential excerpts (hash-table class, /search and /recommend endpoint handlers) are included in the main report. The complete source code is maintained in the project's version-controlled repository.

Appendix D – Datasheets

Not applicable – this is a software-only capstone project with no physical components.

Appendix E – Experimental / Raw Data

Raw latency measurements (10,000 query log) and offline recommendation evaluation output (precision/recall per test fold) underlying Table 11 and Section 4.5.

Appendix F – Ethics / Institutional Approval

Not applicable – the project used a synthetic/public sample dataset with no personally identifiable real-user data; no institutional ethics approval was required.

Appendix G – Project Meeting / Progress Records

Weekly progress notes against the milestone schedule in Section 4.8.

Appendix H – User/Industry Feedback

Not applicable at prototype stage – planned for the production-deployment future-work item in Section 5.2.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not applicable at the time of this report.

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement (front matter) and Table 13 (Roles and Contribution, Section 4.8).

CAPSTONE PROJECT OUTCOME MAPPING SHEET

(To be completed by the department / faculty assessor)

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)